

What if the deployed user for the application speaks another language? This generates the need to internationalise the app. You have to focus on how to localise its values which primarily includes texts and layouts considering each locale value for the application support. For such cases, there are widgets and tools are provided by Flutter which helps in the internationalisation as their libraries are enriched with the information.

In this segment, you have to focus on three fundamentals which include-

- Defining the locales on application support
- Managing the locales app-specific values
- To keep track over the device locale's

There is an internationalised package provided by Flutter (by default) which is US English localisation. If you are looking for the specifications in other languages as well then you need to mention the additional material app properties. Along with it, a separate package must have to be introduced which is called as `Flutter_localizations`. From April this year, this package has introduced the support in 52 languages. If you also want the application to run smoothly on your platform then it is mandatory to add the package "`flutter_cupertino_localization`".

In order to use this localisation, you have to add the package as a dependency in your `pubspec.yaml` file.

Next, you have to import the library named "`flutter_localization`" and then specify the localisation delegates and the supportedLocales for the material app detailing. The applications which are based on `WidgetsApps` are almost similar (except the `GlobalMaterialLocalizations.delegate`)

Advanced Locale

For some languages, in order to define multiple instances, a language code is required for efficient differentiation. Say a fully differentiated language variant consists of the language code, script code, and associated country code. It is because of the extensibility of general script as well as the regional differences in the manner of characters. These characters have to follow the same script type. In order to mention every single variant, the country Code must be followed and the list of supported locales must definitely be included in the detailing.

This will make sure that your application can differentiate between the localised content and all the various combinations associated in the following countries. In case the user's preferred locale is not mentioned, then under the circumstances the closest match can be used. Here, Flutter